

**VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF SCIENCE**

Faculty of Information Technology



**REPORT
LAB 1: HDFS**

COURSE: INTRODUCTION TO BIG DATA

LECTURER:

- Nguyen Ngoc Thao
- Huynh Lam Hai Dang
- Tran Huy Ban

CLASS: 23KHDL2

STUDENT INFORMATION:

- Bui Nam Viet - Student ID: 23127516

TP.HCM, July 10, 2026.

Contents

1	HADOOP PSEUDO-DISTRIBUTED CLUSTER SETUP (SINGLE NODE)	4
1.1	Java Environment Setup (OpenJDK 17)	4
1.1.1	Step 1: Update package lists and system packages	4
1.1.2	Step 2: Install Java Development Kit (OpenJDK 17 JDK)	5
1.1.3	Step 3: Verify installation success (Verification)	5
1.2	Creating a Dedicated User and Configuring Passwordless SSH	5
1.2.1	Step 1: Create user and grant administrative privileges	6
1.2.2	Step 2: Install OpenSSH service	6
1.2.3	Step 3: Generate RSA key pair (SSH Key Generation)	7
1.2.4	Step 4: Authorize key and configure file system permissions	7
1.2.5	Step 5: Verify successful Passwordless SSH connection	8
1.3	Downloading, Installing, and Configuring Environment Variables for Apache Hadoop	9
1.3.1	Step 1: Download Apache Hadoop package	9
1.3.2	Step 2: Extract and organize directory structure	9
1.3.3	Step 3: Assign ownership to target directory	10
1.3.4	Step 4: Configure environment variables via <code>.bashrc</code>	10
1.3.5	Step 5: Activate environment variables	10
1.4	Configuring Java Path in <code>hadoop-env.sh</code> and System Verification	11
1.4.1	Step 1: Set Java path in <code>hadoop-env.sh</code>	11
1.4.2	Step 2: System version verification	11
1.5	Configuring Core XML Files and Starting the Single-Node Hadoop Cluster	12
1.5.1	Step 1: Configure <code>core-site.xml</code>	12
1.5.2	Step 2: Configure <code>hdfs-site.xml</code>	12
1.5.3	Step 3: Configure <code>mapred-site.xml</code> and <code>yarn-site.xml</code>	13
1.5.4	Step 4: Format the NameNode (HDFS Format)	13
1.5.5	Step 5: Start all services and verify system status	14
1.6	Cluster Status Verification via Web UI	14
1.6.1	Step 1: Explore NameNode Web UI	14
1.6.2	Step 2: Explore YARN ResourceManager Web UI	15
2	Pseudo-Distributed Mode	16
2.1	Step 1: Creating the System Directory and Dedicated User	16
2.2	Step 2: Managing Subdirectories, Uploading Data, and Configuring Permissions	16
2.3	Step 3: Executing the JAR Program for Automated System Verification	17
3	Word Count	17
3.1	Step 1: Creating the HDFS Directory Structure and Uploading the Input Dataset	17
3.2	Step 2: Compiling the Scala Application	18
3.3	Step 3: Submitting the Job to YARN and Resolving the <code>ClassNotFoundException</code>	18
3.4	Step 4: Successful Job Execution	19
4	Fully Distributed Hadoop Cluster on Virtual Machines	20
4.1	System Architecture	20
4.2	Deployment Procedure	21
4.2.1	Step 1: Creating the Internal Docker Network	21

4.2.2	Step 2: Building the Base Docker Image	21
4.2.3	Step 3: Launching the Hadoop Virtual Machines	21

1 HADOOP PSEUDO-DISTRIBUTED CLUSTER SETUP (SINGLE NODE)

1.1 Java Environment Setup (OpenJDK 17)

Hadoop runs on the Java platform, so a compatible Java environment must be installed before the cluster is configured. This lab uses **OpenJDK 17** on WSL 2 (Ubuntu).

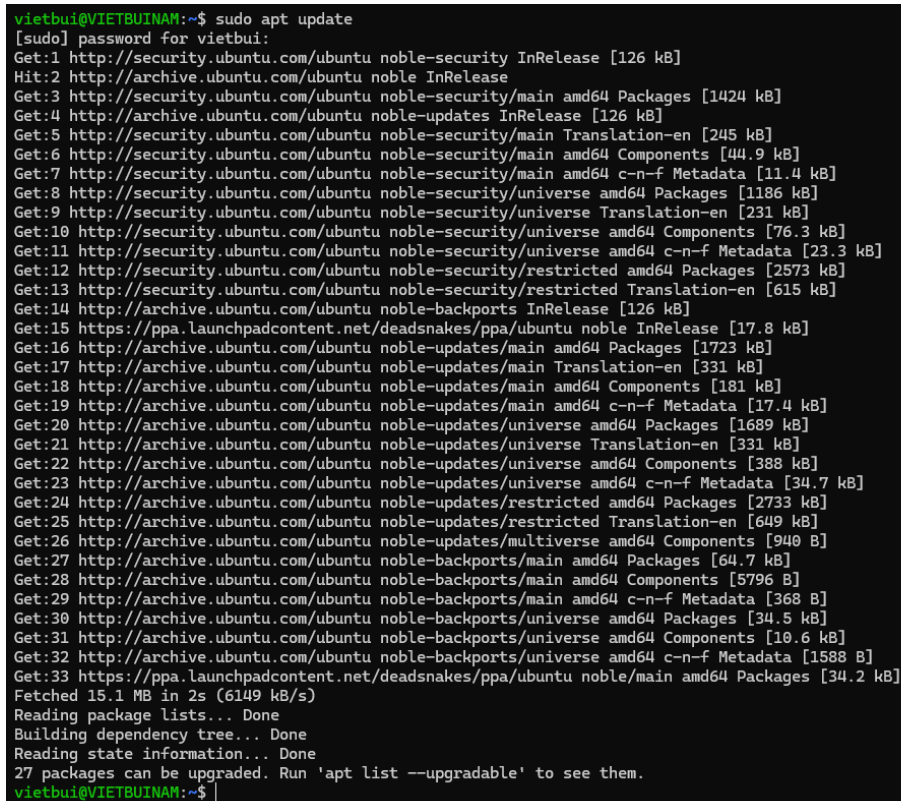
The setup was completed as follows:

1.1.1 Step 1: Update package lists and system packages

The Ubuntu package index was updated, and installed packages were upgraded before installing Hadoop dependencies:

```
sudo apt update
sudo apt upgrade -y
```

Explanation: apt update refreshes package metadata, while apt upgrade -y installs available updates without additional prompts.



```
vietchui@VIETBUINAM:~$ sudo apt update
[sudo] password for vietchui:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1424 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [245 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [44.9 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [11.4 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1186 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [231 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [76.3 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [23.3 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2573 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [615 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:15 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu noble InRelease [17.8 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1723 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [331 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [17.4 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1689 kB]
Get:21 http://archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [331 kB]
Get:22 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [388 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [34.7 kB]
Get:24 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2733 kB]
Get:25 http://archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [649 kB]
Get:26 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:27 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [64.7 kB]
Get:28 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5796 B]
Get:29 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 c-n-f Metadata [368 B]
Get:30 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [34.5 kB]
Get:31 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.6 kB]
Get:32 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 c-n-f Metadata [1588 B]
Get:33 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu noble/main amd64 Packages [34.2 kB]
Fetched 15.1 MB in 2s (6149 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
27 packages can be upgraded. Run 'apt list --upgradable' to see them.
vietchui@VIETBUINAM:~$
```

Figure 1: System updating on WSL Ubuntu.

```

hadoop@VIETBUINAM:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
Get more security updates through Ubuntu Pro with 'esm-apps' enabled:
python3-pip-whl python3-wheel python3-pip
Learn more about Ubuntu Pro at https://ubuntu.com/pro
The following packages will be upgraded:
  apparmor cloud-init iproute2 libapparmor1 libcups2t64 libnss-systemd libpam-systemd libssl3t64 libsystemd-shared libsystemd0 libudev1 libxmlb2 nano openssl rsync snapd systemd systemd-dev systemd-resolved
  systemt-sysv systemd-timesyncd udev xdg-user-dirs xdg-utils xz-utils xz-utils-bin xz-utils-dev xz-utils-static xz-utils-static-dev xz-utils-static-dev-static
27 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
21 standard LTS security updates
Need to get 68.8 MB of archives.
After this operation, 325 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libnss-systemd amd64 255.4-ubuntu0.16 [159 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-dev all 255.4-ubuntu0.16 [106 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-timesyncd amd64 255.4-ubuntu0.16 [35.3 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-resolved amd64 255.4-ubuntu0.16 [596 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libsystemd-shared amd64 255.4-ubuntu0.16 [2076 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libsystemd0 amd64 255.4-ubuntu0.16 [312 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-sysv amd64 255.4-ubuntu0.16 [11.9 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libpam-systemd amd64 255.4-ubuntu0.16 [225 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd amd64 255.4-ubuntu0.16 [3479 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 udev amd64 255.4-ubuntu0.16 [3875 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libudev1 amd64 255.4-ubuntu0.16 [177 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libssl3t64 amd64 3.0.13-ubuntu0.11 [1042 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libapparmor2 amd64 4.0.1-0ubuntu0.24.04.7 [51.3 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 rsync amd64 3.2.7-ubuntu0.5 [403 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 iproute2 amd64 6.1.0-ubuntu0.2 [1120 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 openssl amd64 3.0.13-ubuntu0.2 [1383 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 xz-utils amd64 2.9.1-0ubuntu0.24.04.7 [1883 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 nano amd64 2.9.1-0ubuntu0.24.04.7 [608 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 xz-utils-bin amd64 2.9.1-0ubuntu0.24.04.7 [608 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 xz-utils-static amd64 2.9.1-0ubuntu0.24.04.7 [608 kB]
Get:21 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 xz-utils-static-dev amd64 2.9.1-0ubuntu0.24.04.7 [608 kB]
Get:22 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 xz-utils-static-dev-static amd64 2.9.1-0ubuntu0.24.04.7 [608 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 xz-utils-static-dev-static-dev amd64 2.9.1-0ubuntu0.24.04.7 [608 kB]
Get:24 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 xz-utils-static-dev-static-dev-static amd64 2.9.1-0ubuntu0.24.04.7 [608 kB]
Get:25 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 xz-utils-static-dev-static-dev-static-dev amd64 2.9.1-0ubuntu0.24.04.7 [608 kB]
Get:26 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 xz-utils-static-dev-static-dev-static-dev-static amd64 2.9.1-0ubuntu0.24.04.7 [608 kB]
Get:27 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 xz-utils-static-dev-static-dev-static-dev-static-dev amd64 2.9.1-0ubuntu0.24.04.7 [608 kB]
Fetched 68.8 MB in 4s (16.9 MB/s)

```

Figure 2: System upgrading on WSL Ubuntu.

1.1.2 Step 2: Install Java Development Kit (OpenJDK 17 JDK)

After the system update, the full OpenJDK 17 JDK package was installed:

```
sudo apt install openjdk-17-jdk -y
```

Explanation: The -y flag accepts the installation and required dependencies automatically.

```

hadoop@VIETBUINAM:~$ sudo apt install openjdk-17-jdk -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  alsa-topology-conf alsa-ucm-conf ca-certificates-java fonts-dejavu-extra java-common libasound2-data libasound2t64
  libatk-wrapper-java libatk-wrapper-java-jni libglib2.0-0 libice-dev libice6 libnss3 libnss3-dev libpcsc-lite1
  libpthread-stubs0-dev libsm-dev libsm6 libx11-dev libxau-dev libxaw7 libxcb-shape0 libxcb1-dev libxdmcp-dev libxft2
  libxkbfile1 libxmu6 libxmu-dev libxmu6t64 libxv1 libxxf86dgal openjdk-17-jdk-headless openjdk-17-jre
  openjdk-17-jre-headless x11-utils x11proto-dev xorg-sgml-doctools xtrans-dev
Suggested packages:
  default-jre alsa-utils libasound2-plugins libice-doc pcscd libsm-doc libx11-doc libxcb-doc libxmu-doc openjdk-17-demo
  openjdk-17-source visualvm libnss-mdns fonts-ipafont-gothic fonts-ipafont-mincho fonts-wqy-microhei
  | fonts-wqy-zenhei fonts-indic mesa-utils
Recommended packages:
  luit
The following NEW packages will be installed:
  alsa-topology-conf alsa-ucm-conf ca-certificates-java fonts-dejavu-extra java-common libasound2-data libasound2t64
  libatk-wrapper-java libatk-wrapper-java-jni libglib2.0-0 libice-dev libice6 libnss3 libnss3-dev libpcsc-lite1
  libpthread-stubs0-dev libsm-dev libsm6 libx11-dev libxau-dev libxaw7 libxcb-shape0 libxcb1-dev libxdmcp-dev libxft2
  libxkbfile1 libxmu6 libxmu-dev libxmu6t64 libxv1 libxxf86dgal openjdk-17-jdk openjdk-17-jdk-headless openjdk-17-jre
  openjdk-17-jre-headless x11-utils x11proto-dev xorg-sgml-doctools xtrans-dev
0 upgraded, 39 newly installed, 0 to remove and 0 not upgraded.

```

Figure 3: Java installation on WSL Ubuntu.

1.1.3 Step 3: Verify installation success (Verification)

The Java installation was verified by checking the runtime version:

```
java -version
```

```

hadoop@VIETBUINAM:~$ java -version
openjdk version "17.0.19" 2023-04-21
OpenJDK Runtime Environment (build 17.0.19+10-1-24.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.19+10-1-24.04.2-Ubuntu, mixed mode, sharing)

```

Figure 4: Java version verification on WSL Ubuntu.

- **Result:** The system displayed `openjdk version "17.0.19"` with the 64-Bit Server VM, confirming that Java was installed correctly and ready for Hadoop configuration.

1.2 Creating a Dedicated User and Configuring Passwordless SSH

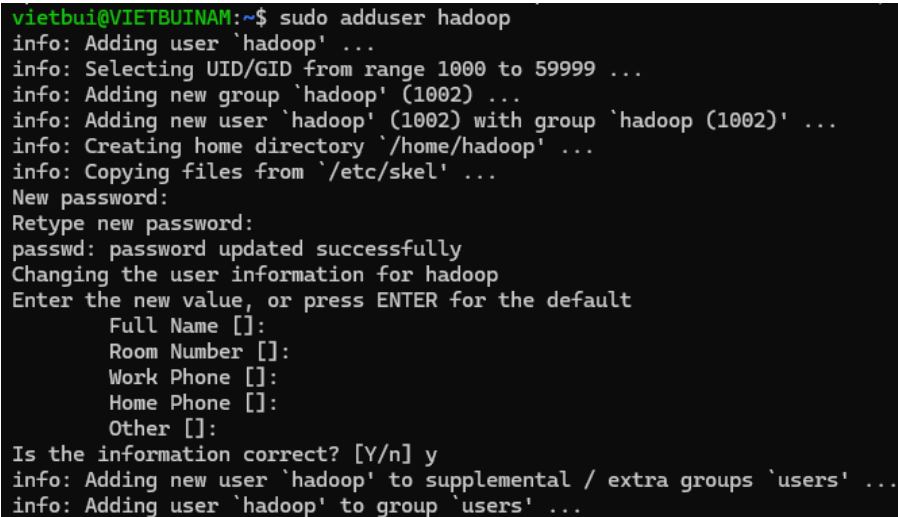
Hadoop uses SSH to manage cluster processes, even in pseudo-distributed single-node mode. A dedicated `hadoop` user was created, and passwordless SSH was configured to support automated service startup.

1.2.1 Step 1: Create user and grant administrative privileges

A separate Linux user was created for the Hadoop environment and granted administrative privileges:

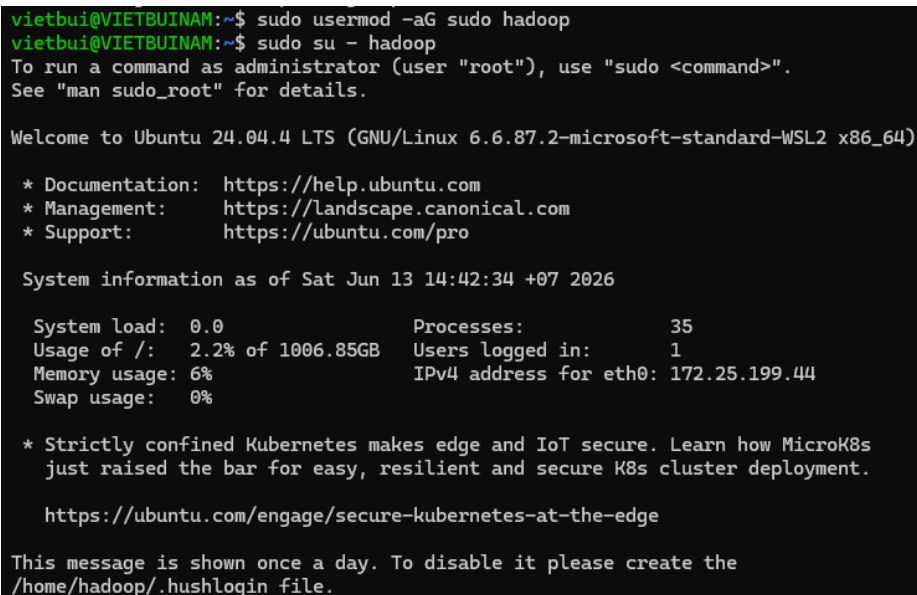
```
sudo adduser hadoop
sudo usermod -aG sudo hadoop
```

Explanation: The first command creates the /home/hadoop account environment. The second command adds the user to the `sudo` group, allowing administrative commands when required. The session was then switched to the new user using `sudo su - hadoop`.



```
vietchui@VIETBUINAM:~$ sudo adduser hadoop
info: Adding user `hadoop' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `hadoop' (1002) ...
info: Adding new user `hadoop' (1002) with group `hadoop (1002)' ...
info: Creating home directory `/home/hadoop' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for hadoop
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
info: Adding new user `hadoop' to supplemental / extra groups `users' ...
info: Adding user `hadoop' to group `users' ...
```

Figure 5: Creating a new user in WSL Ubuntu.



```
vietchui@VIETBUINAM:~$ sudo usermod -aG sudo hadoop
vietchui@VIETBUINAM:~$ sudo su - hadoop
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jun 13 14:42:34 +07 2026

System load:  0.0          Processes:    35
Usage of /:   2.2% of 1006.85GB  Users logged in: 1
Memory usage: 6%          IPv4 address for eth0: 172.25.199.44
Swap usage:  0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/hadoop/.hushlogin file.
```

Figure 6: Granting sudo privileges and switching to hadoop user workspace.

1.2.2 Step 2: Install OpenSSH service

The SSH service package was installed to enable secure local communication:

```
sudo apt install ssh
```

```
hadoop@VIETBUINAM:~$ sudo apt install ssh
[sudo] password for hadoop:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libwrap0 ncurses-term openssh-server openssh-sftp-server ssh-import-id
Suggested packages:
  molly-guard monkeysphere ssh-askpass ufw
The following NEW packages will be installed:
  libwrap0 ncurses-term openssh-server openssh-sftp-server ssh ssh-import-id
0 upgraded, 6 newly installed, 0 to remove and 0 not upgraded.
Need to get 884 kB of archives.
After this operation, 6915 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-sftp-server amd64 1:9.6p1-3ubuntu13.16 [37.3 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 libwrap0 amd64 7.6.q-33 [47.9 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-server amd64 1:9.6p1-3ubuntu13.16 [510 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/main amd64 ncurses-term all 6.4+20240113-1ubuntu2 [275 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 ssh all 1:9.6p1-3ubuntu13.16 [4660 B]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 ssh-import-id all 5.11-0ubuntu2.24.04.1 [10.1 kB]
Fetched 884 kB in 1s (1413 kB/s)
```

Figure 7: Installing OpenSSH service on WSL Ubuntu.

1.2.3 Step 3: Generate RSA key pair (SSH Key Generation)

An RSA key pair was generated for passwordless authentication:

```
ssh-keygen -t rsa
```

Note: The default key location was used, and the passphrase was left empty so Hadoop services could connect without manual input.

```
hadoop@VIETBUINAM:~$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/hadoop/.ssh/id_rsa):
Created directory '/home/hadoop/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/hadoop/.ssh/id_rsa
Your public key has been saved in /home/hadoop/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:TdPk5Ee+SJEJtArop/NUDESvh1fvdoKd0HBQR84S4U hadoop@VIETBUINAM
The key's randomart image is:
+---[RSA 3072]-----+
|
|.oo.oo==oB*+|
| . o o*=E o=|
| .ooo*.*.o|
| .oo.o +.+|
| .oS.. . +.|
| . . . o o|
| . = . o o|
| o o .|
|_|
+----[SHA256]-----+
```

Figure 8: Successful RSA key generation with randomart visualization.

1.2.4 Step 4: Authorize key and configure file system permissions

The generated public key was added to `authorized_keys`, file permissions were configured, and the SSH service was started:

```
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
sudo chmod 640 ~/.ssh/authorized_keys
sudo service ssh start
```

```
hadoop@VIETBUINAM:~$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
hadoop@VIETBUINAM:~$ sudo chmod 640 ~/.ssh/authorized_keys
hadoop@VIETBUINAM:~$ sudo service ssh start
hadoop@VIETBUINAM:~$ ssh localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:ObwPROMV4I9Dab0FGtVmSiVyGm/FD1vKCysW05J98hw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jun 13 14:46:18 +07 2026

System load:  0.0                Processes:    38
Usage of /:   2.2% of 1006.85GB   Users logged in: 1
Memory usage: 6%                IPv4 address for eth0: 172.25.199.44
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.
```

Figure 9: Adding the public key to `authorized_keys` and starting the SSH service.

Issue encountered: Incorrect permissions on SSH authorized keys

During passwordless SSH configuration, the command

```
sudo chmod 640 ~/.ssh/authorized_keys
```

caused the SSH server to reject public-key authentication. Although the mode appeared restrictive, using `sudo` with 640 produced an ownership and permission state that SSH did not accept for `authorized_keys`.

SSH requires strict ownership and permission settings for both the `~/.ssh` directory and the `authorized_keys` file. If these settings are too permissive or assigned to the wrong owner, the SSH daemon disables public-key authentication for security.

The permissions were corrected under the `hadoop` account without using `sudo`:

```
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

The mode 700 restricts access to the `.ssh` directory to the owner, and 600 restricts `authorized_keys` to owner read/write access. After this correction, SSH accepted public-key authentication.

1.2.5 Step 5: Verify successful Passwordless SSH connection

The SSH configuration was tested by connecting to localhost:

```
ssh localhost
```



```

hadoop@VIETBUINAM:~$ chmod 700 ~/.ssh
hadoop@VIETBUINAM:~$ chmod 600 ~/.ssh/authorized_keys
hadoop@VIETBUINAM:~$ ssh localhost
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.6.87.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jun 13 14:46:18 +07 2026

System load:  0.0               Processes:    38
Usage of /:   2.2% of 1006.85GB Users logged in: 1
Memory usage: 6%               IPv4 address for eth0: 172.25.199.44
Swap usage:  0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge
Last login: Sat Jun 13 14:46:18 2026 from 127.0.0.1

```

Figure 10: Typing **yes** to confirm the host fingerprint and successful passwordless login.

- **Result:** On the first connection, the system displayed a host fingerprint prompt. After entering **yes**, the host was stored in **known_hosts**, and the SSH client logged into **hadoop@VIETBUINAM** without requesting a password.

1.3 Downloading, Installing, and Configuring Environment Variables for Apache Hadoop

After Java and SSH were configured, the Apache Hadoop distribution was downloaded and environment variables were set so Hadoop commands could run from any working directory.

1.3.1 Step 1: Download Apache Hadoop package

The Hadoop binary distribution was downloaded from the official Apache repository:

```
wget https://downloads.apache.org/hadoop/common/stable/hadoop-3.5.0.tar.gz
```

Explanation: The file **hadoop-3.5.0.tar.gz** contains the Hadoop components required for HDFS and YARN in this lab.

```

hadoop@VIETBUINAM:~$ wget https://downloads.apache.org/hadoop/common/stable/hadoop-3.5.0.tar.gz
--2026-06-13 14:51:24-- https://downloads.apache.org/hadoop/common/stable/hadoop-3.5.0.tar.gz
Resolving downloads.apache.org (downloads.apache.org)... 88.99.208.237, 95.216.224.44, 2a01:4f8:10a:39da::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|88.99.208.237|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 581280703 (554M) [application/x-gzip]
Saving to: 'hadoop-3.5.0.tar.gz'

hadoop-3.5.0.tar.gz      100%[=====] 554.35M  16.5MB/s   in 37s
2026-06-13 14:52:00 (15.1 MB/s) - 'hadoop-3.5.0.tar.gz' saved [581280703/581280703]

```

Figure 11: Downloading Apache Hadoop package.

1.3.2 Step 2: Extract and organize directory structure

The downloaded archive was extracted and renamed for simpler path configuration:

```
tar -xzf hadoop-3.5.0.tar.gz
sudo mv hadoop-3.5.0 hadoop
```

Explanation: **tar -xzf** extracts the archive, and **mv** renames the extracted directory to **hadoop** in the user's home directory.

```

hadoop@VIETBUINAM:~$ tar -xzf hadoop-3.5.0.tar.gz
hadoop@VIETBUINAM:~$ sudo mv hadoop-3.5.0 hadoop
[sudo] password for hadoop:

```

Figure 12: Extracting and renaming Hadoop directory.

1.3.3 Step 3: Assign ownership to target directory

Ownership of the Hadoop directory was assigned to the dedicated `hadoop` user:

```
sudo chown -R hadoop:hadoop ~/hadoop
```

Explanation: The recursive `-R` option applies ownership to all files and subdirectories, preventing permission errors during service startup.

```
hadoop@VIETBUINAM:~$ ls
hadoop  hadoop-3.5.0.tar.gz
hadoop@VIETBUINAM:~$ sudo chown -R hadoop:hadoop ~/hadoop
hadoop@VIETBUINAM:~$ ls -l
total 567668
drwxr-xr-x 11 hadoop hadoop      4096 Mar 25 00:31 hadoop
-rw-rw-r-- 1 hadoop hadoop 581280703 Apr  2 03:43 hadoop-3.5.0.tar.gz
hadoop@VIETBUINAM:~$ |
```

Figure 13: Verifying ownership using `ls -l` showing `hadoop:hadoop`.

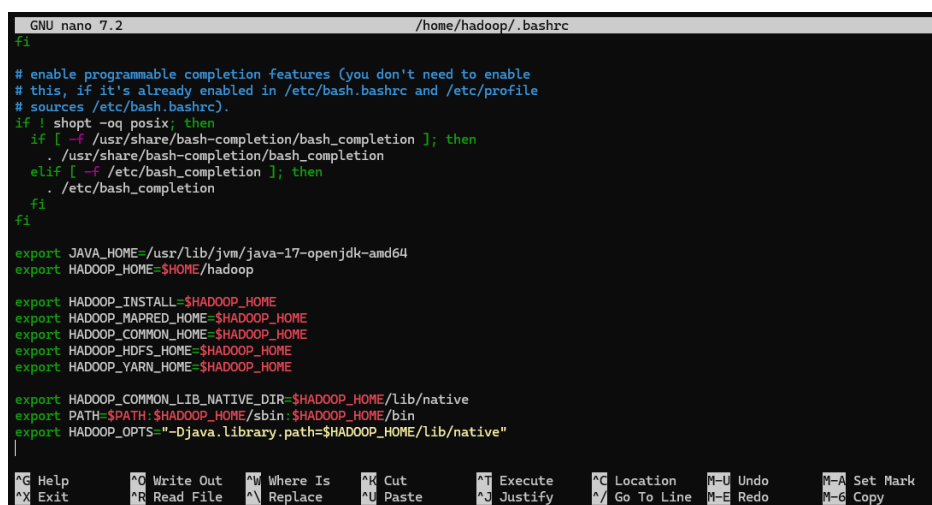
1.3.4 Step 4: Configure environment variables via `.bashrc`

The `.bashrc` file was edited to define Hadoop and Java environment variables:

```
nano ~/.bashrc
```

The following configuration was appended to the file:

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
export HADOOP_HOME=$HOME/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"
```



The screenshot shows the nano 7.2 text editor editing the file `/home/hadoop/.bashrc`. The file content includes comments about bash completion and the Hadoop environment variable exports. The terminal window title is `GNU nano 7.2 /home/hadoop/.bashrc`. The bottom status bar shows various nano editor controls like Help, Exit, Write Out, Read File, Where Is, Replace, Cut, Paste, Execute, Justify, Location, Go To Line, Undo, Redo, Set Mark, and Copy.

Figure 14: Editing `.bashrc` with Hadoop environment variables in nano.

1.3.5 Step 5: Activate environment variables

After saving the file in nano, the current shell session was reloaded:

```
source ~/.bashrc
```

- **Result:** The OpenJDK 17 and Apache Hadoop 3.5.0 paths were loaded into the shell environment. Commands such as `hdfs`, `hadoop`, and `start-dfs.sh` can now run from the `hadoop` user session.

1.4 Configuring Java Path in `hadoop-env.sh` and System Verification

Hadoop must reference the Java installation in its internal environment script so HDFS, YARN, and MapReduce services can start consistently.

1.4.1 Step 1: Set Java path in `hadoop-env.sh`

The `hadoop-env.sh` file in the Hadoop configuration directory was opened:

```
nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
```

The `JAVA_HOME` setting was added for the installed OpenJDK 17 path:

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

```
GNU nano 7.2 /home/hadoop/hadoop/etc/hadoop/hadoop-env.sh
##
## {YARN_xyz|HDFS_xyz} > HADOOP_xyz > hard-coded defaults
##
# Many of the options here are built from the perspective that users
# may want to provide OVERWRITING values on the command line.
# For example:
#
#   JAVA_HOME=/usr/java/testing hdfs dfs -ls
#
# Therefore, the vast majority (BUT NOT ALL!) of these defaults
# are configured for substitution and not append. If append
# is preferable, modify this file accordingly.
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
###
# Generic settings for HADOOP
###
# Technically, the only required environment variable is JAVA_HOME.
# All others are optional. However, the defaults are probably not
# preferred. Many sites configure these options outside of Hadoop,
# such as in /etc/profile.d
#
# The java implementation to use. By default, this environment
# variable is REQUIRED on ALL platforms except OS X!
# export JAVA_HOME=
```

Figure 15: Editing `hadoop-env.sh` with Java path in nano.

1.4.2 Step 2: System version verification

The Hadoop version command was executed to verify the framework installation and environment configuration:

```
hadoop version
```

```
hadoop@VIETBUINAM:~$ sudo nano $HADOOP_HOME/etc/hadoop/hadoop-env.sh
hadoop@VIETBUINAM:~$ hadoop version
Hadoop 3.5.0
Source code repository https://github.com/apache/hadoop.git -r dbcc7cd797100e6b32cd84f85b53a5193a5f9af0
Compiled by cnauroth on 2026-03-24T16:56Z
Compiled on platform linux-x86_64
Compiled with protoc 3.25.5
From source with checksum 64582ed0a62a169c39a19c734a8f33b5
This command was run using /home/hadoop/hadoop/share/hadoop/common/hadoop-common-3.5.0.jar
```

Figure 16: Output of `hadoop version` showing Apache Hadoop 3.5.0 core information.

- **Result:** The command returned Hadoop 3.5.0 information and the `hadoop-common-3.5.0.jar` path, confirming that the base Hadoop environment was configured correctly.

1.5 Configuring Core XML Files and Starting the Single-Node Hadoop Cluster

Pseudo-distributed mode requires the core Hadoop XML files to define HDFS access, local storage paths, MapReduce execution, and YARN services.

1.5.1 Step 1: Configure core-site.xml

core-site.xml was configured to define the default HDFS URI and communication port:

```
nano $HADOOP_HOME/etc/hadoop/core-site.xml
```

Inserted property: The default URI was set to `hdfs://localhost:9000`.

```
GNU nano 7.2 /home/hadoop/hadoop/etc/hadoop/core-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
Licensed under the Apache License, Version 2.0 (the "License");
you may not use this file except in compliance with the License.
You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

Unless required by applicable law or agreed to in writing, software
distributed under the License is distributed on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
See the License for the specific language governing permissions and
limitations under the License. See accompanying LICENSE file.
-->
<!-- Put site-specific property overrides in this file. -->

<configuration>
<property>
  <name>fs.defaultFS</name>
  <value>hdfs://localhost:9000</value>
</property>
</configuration>
```

Figure 17: Configuring default filesystem property in core-site.xml.

1.5.2 Step 2: Configure hdfs-site.xml

Local directories were created for NameNode metadata and DataNode block storage, then added to hdfs-site.xml:

```
mkdir -p /home/hadoop/hdfs/{namenode,datanode}
nano $HADOOP_HOME/etc/hadoop/hdfs-site.xml
```

```
GNU nano 7.2 /home/hadoop/hadoop/etc/hadoop/hdfs-site.xml
<configuration>
<property>
  <name>dfs.replication</name>
  <value>1</value>
</property>

<property>
  <name>dfs.namenode.name.dir</name>
  <value>file:///home/hadoop/hdfs/namenode</value>
</property>

<property>
  <name>dfs.datanode.data.dir</name>
  <value>file:///home/hadoop/hdfs/datanode</value>
</property>
</configuration>
```

Figure 18: Configuring HDFS storage directories and replication factor.

Inserted properties: The replication factor `dfs.replication` was set to 1 for the single-

node setup, and the NameNode and DataNode storage paths were set to the created directories.

1.5.3 Step 3: Configure `mapred-site.xml` and `yarn-site.xml`

MapReduce was configured to use YARN for job scheduling and resource management:

```
nano $HADOOP_HOME/etc/hadoop/mapred-site.xml
nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
```

Inserted properties: In `mapred-site.xml`, the execution framework was set to yarn. In `yarn-site.xml`, `mapreduce_shuffle` was enabled to support the MapReduce Shuffle & Sort phase.

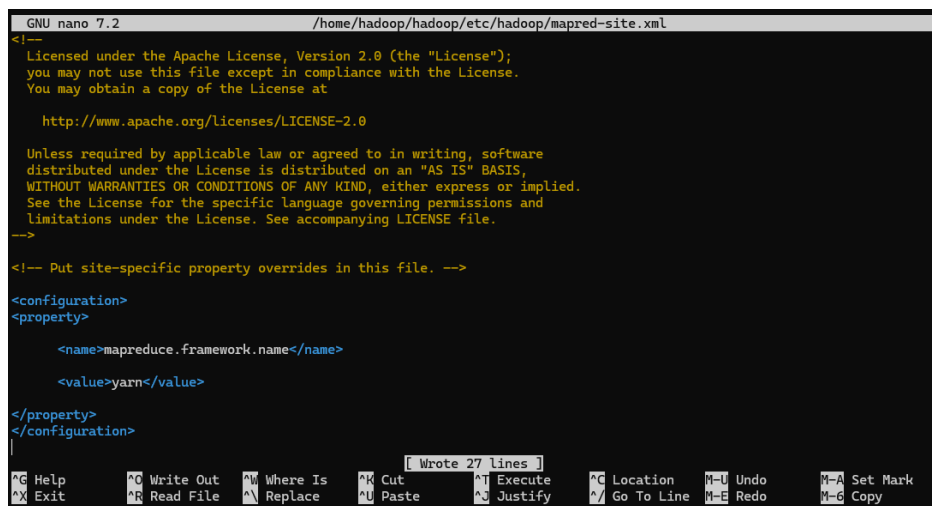
A screenshot of the GNU nano 7.2 text editor. The title bar shows the file path as /home/hadoop/hadoop/etc/hadoop/mapred-site.xml. The editor contains XML configuration for MapReduce. It starts with a license notice. Then, within a <configuration> block, there is a <property> element with <name>mapreduce.framework.name</name> and <value>yarn</value>. The bottom status bar shows various keyboard shortcuts like ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^U Undo, ^M Set Mark, ^X Exit, ^R Read File, ^N Replace, ^P Paste, ^J Justify, ^_ Go To Line, ^E Redo, and ^G Copy. A message '[Wrote 27 lines]' is also visible.

Figure 19: Configuring YARN framework in `mapred-site.xml`.

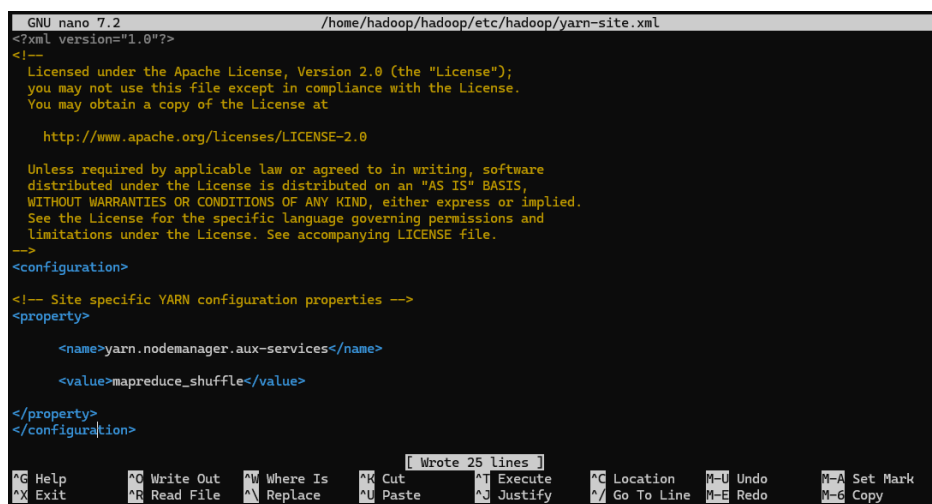
A screenshot of the GNU nano 7.2 text editor. The title bar shows the file path as /home/hadoop/hadoop/etc/hadoop/yarn-site.xml. The editor contains XML configuration for YARN. It starts with a license notice. Then, within a <configuration> block, there is a comment <!-- Site specific YARN configuration properties --> followed by a <property> element with <name>yarn.nodemanager.aux-services</name> and <value>mapreduce_shuffle</value>. The bottom status bar shows various keyboard shortcuts like ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^U Undo, ^M Set Mark, ^X Exit, ^R Read File, ^N Replace, ^P Paste, ^J Justify, ^_ Go To Line, ^E Redo, and ^G Copy. A message '[Wrote 25 lines]' is also visible.

Figure 20: Enabling NodeManager shuffle service in `yarn-site.xml`.

1.5.4 Step 4: Format the NameNode (HDFS Format)

Before the first cluster startup, the NameNode was formatted to initialize HDFS metadata:

```
hdfs namenode -format
```

```

hadoop@VIETBUINAM:~$ sudo nano $HADOOP_HOME/etc/hadoop/mapred-site.xml
hadoop@VIETBUINAM:~$ sudo nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
hadoop@VIETBUINAM:~$ hdfs namenode -format
2026-06-13 15:30:41,039 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG: host = VIETBUINAM/127.0.1.1
STARTUP_MSG: args = [-format]
STARTUP_MSG: version = 3.5.0
STARTUP_MSG: classpath = /home/hadoop/hadoop/etc/hadoop:/home/hadoop/hadoop/share/hadoop/common/lib/slf4j-reload4j-1.7
.36.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jakarta.xml.bind-api-2.3.3.jar:/home/hadoop/hadoop/share/hadoop/comm
on/lib/zookeeper-3.8.6.jar:/home/hadoop/hadoop/share/hadoop/common/lib/nimbus-jose-jwt-10.4.jar:/home/hadoop/hadoop/shar
e/hadoop/common/lib/commons-text-1.14.0.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jetty-io-9.4.58.v20250814.jar:/h
ome/hadoop/hadoop/share/hadoop/common/lib/kerby-pkix-2.0.3.jar:/home/hadoop/hadoop/share/hadoop/common/lib/jspecify-1.0.

```

Figure 21: Formatting the NameNode metadata storage.

1.5.5 Step 5: Start all services and verify system status

The startup scripts were executed, and Java processes were checked to verify the cluster status:

```

start-dfs.sh
start-yarn.sh
jps

```

```

hadoop@VIETBUINAM:~$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [VIETBUINAM]
VIETBUINAM: Warning: Permanently added 'vietbuinam' (ED25519) to the list of known hosts.
hadoop@VIETBUINAM:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
hadoop@VIETBUINAM:~$ jps
7588 DataNode
8148 ResourceManager
8279 NodeManager
7448 NameNode
8697 Jps
7869 SecondaryNameNode
hadoop@VIETBUINAM:~$ |

```

Figure 22: Successfully starting HDFS, YARN and jps showing all core processes.

- **Result:** The jps output showed **NameNode**, **SecondaryNameNode**, **DataNode**, **ResourceManager**, and **NodeManager** running under separate PIDs. This confirms that the pseudo-distributed Hadoop cluster was started successfully.

1.6 Cluster Status Verification via Web UI

Hadoop Web UI pages were used to confirm the HDFS and YARN status after startup.

1.6.1 Step 1: Explore NameNode Web UI

The NameNode Web UI was opened at <http://localhost:9870> to inspect HDFS status.

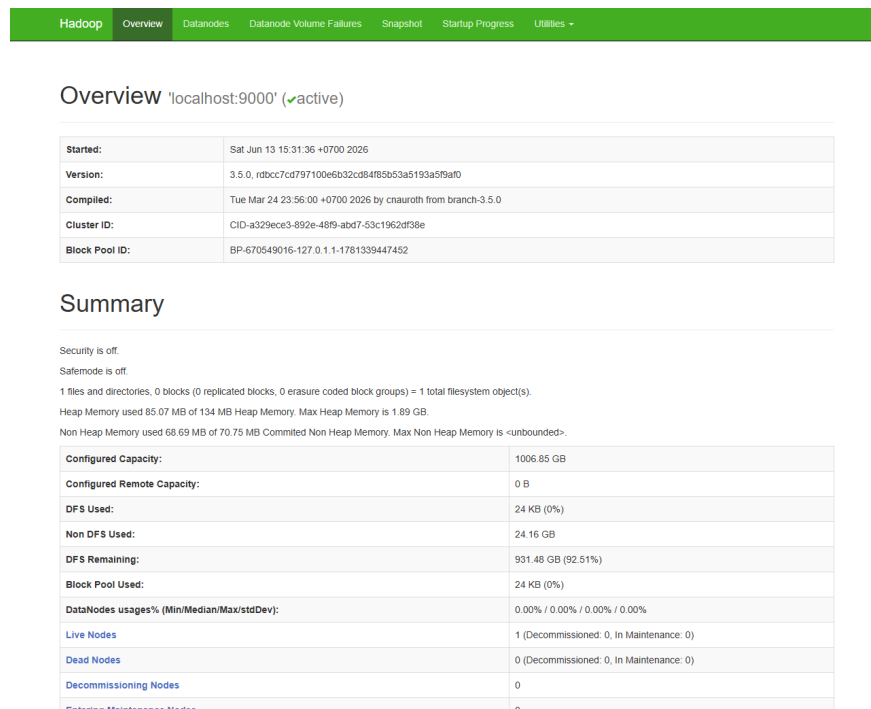


Figure 23: NameNode Web UI showing cluster status.

- **System Metrics Analysis:**

- **State:** The interface showed the NameNode state as **active** and reported Hadoop version **3.5.0**, matching the installed environment.
- **Summary:** The status **Safemode is off** indicates that HDFS is ready for read/write operations. The **Live Nodes** value was **1**, confirming that the single DataNode was connected, and **DFS Remaining** was **92.51%**.

1.6.2 Step 2: Explore YARN ResourceManager Web UI

The ResourceManager Web UI was opened at `http://localhost:8088` to inspect YARN status.

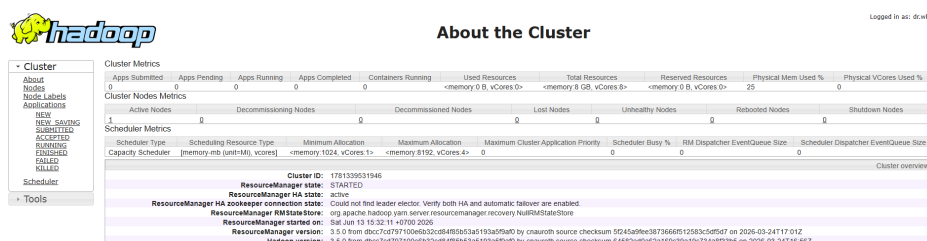


Figure 24: YARN ResourceManager Web UI showing cluster metrics and scheduler information.

- **System Metrics Analysis:**

- **Cluster Metrics:** `Apps Running` and `Apps Completed` were both `0` because no processing jobs had been submitted. The `Active Nodes` value was `1`, indicating that the `NodeManager` was available for containers.
- **Total Resources:** The interface reported `8 GB` virtual memory and `8 vCores`. The `ResourceManager HA` state was `active`, confirming that the scheduler was initialized.

- **Conclusion:** The Web UI metrics matched the local process verification, confirming

that the cluster was ready for HDFS operations through the CLI.

2 Pseudo-Distributed Mode

2.1 Step 1: Creating the System Directory and Dedicated User

- **Create the root directory:** The parent directory `/hcmus` was created on HDFS as the common storage workspace:

```
hdfs dfs -mkdir /hcmus
```

- **Create a dedicated system user:** A Linux user named `khtn_23127516` was created for ownership assignment:

```
sudo adduser khtn_23127516
```

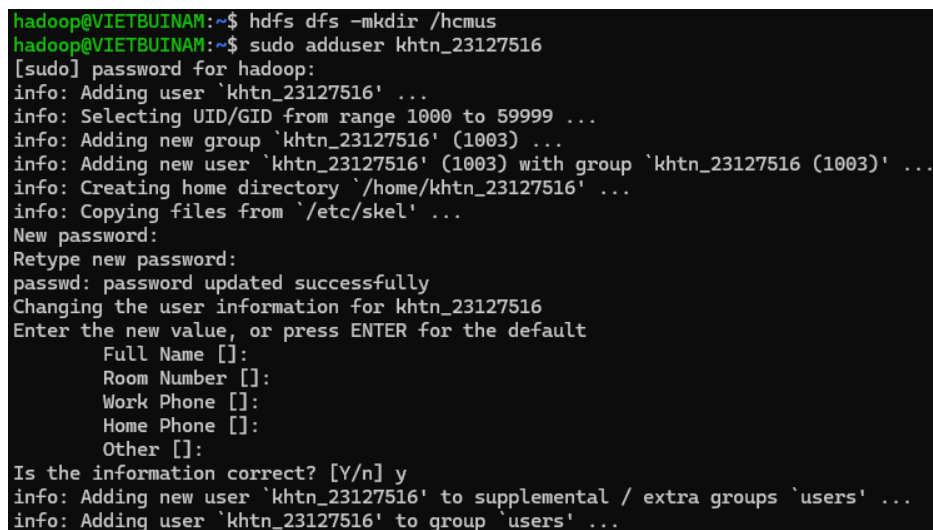
A terminal window showing the execution of commands to create a user. The user is at a prompt on a machine named VIETBUINAM. They run 'hdfs dfs -mkdir /hcmus' and then 'sudo adduser khtn_23127516'. The 'adduser' command prompts for a password, then shows progress messages: 'Adding user', 'Selecting UID/GID from range 1000 to 59999', 'Adding new group', 'Adding new user', 'Creating home directory', and 'Copying files from /etc/skel'. It then asks for a new password and to retype it. After confirming the password, it asks for user information (Full Name, Room Number, Work Phone, Home Phone, Other) and whether the information is correct. Finally, it adds the user to the 'users' group and adds the user to the supplemental / extra groups 'users'.

Figure 25: Creating the root directory `/hcmus` and configuring the user `khtn_23127516`.

2.2 Step 2: Managing Subdirectories, Uploading Data, and Configuring Permissions

- **Create the student directory:** The subdirectory `/hcmus/23127516` was created under the HDFS root directory hierarchy.
- **Upload data to HDFS:** A local text file named `sample.txt` was created and uploaded to HDFS:

```
echo "HDFS_Test_File" > sample.txt
hdfs dfs -put sample.txt /hcmus/23127516/
```

- **Configure permissions and transfer ownership:** Permission mode `744` was assigned, and ownership of the student directory was transferred to `khtn_23127516`:

```
hdfs dfs -chmod 744 /hcmus/23127516
hdfs dfs -chown khtn_23127516 /hcmus/23127516
```



```

hadoop@VIETBUINAM:~$ hdfs dfs -mkdir /hcmus/23127516
hadoop@VIETBUINAM:~$ echo "HDFS Test File" > sample.txt
hadoop@VIETBUINAM:~$ hdfs dfs -put sample.txt /hcmus/23127516/
hadoop@VIETBUINAM:~$ hdfs dfs -chmod 744 /hcmus/23127516
hadoop@VIETBUINAM:~$ hdfs dfs -chown khtn_23127516 /hcmus/23127516
hadoop@VIETBUINAM:~$ hdfs dfs -ls /hcmus
Found 1 items
drwxr--r--  - khtn_23127516 supergroup          0 2026-06-27 10:42 /hcmus/23127516

```

Figure 26: Executing data operations, applying chmod 744, and verifying the directory structure using the ls command.

- **Verification:** The command `hdfs dfs -ls /hcmus` confirmed the expected state. The directory showed permission attribute `drwxr--r--`, corresponding to mode 744, and ownership was assigned to `khtn_23127516`.

2.3 Step 3: Executing the JAR Program for Automated System Verification

The verification JAR was executed from `/home/hadoop/run` to validate the HDFS configuration automatically.

- **Execution command:**

```

mkdir -p ~/run && cd ~/run
java -jar ~/hadoop-test.jar 9000 /hcmus/23127516

```

```

hadoop@VIETBUINAM:~$ mkdir -p ~/run
hadoop@VIETBUINAM:~$ cd ~/run
hadoop@VIETBUINAM:~/run$ java -jar ~/hadoop-test.jar 9000 /hcmus/23127516
Trying to read /hcmus/23127516
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23127516/sample.txt
Your student ID: 23127516 (ensure it matches your student ID)
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
File written at /home/hadoop/run/23127516_verification.txt

```

Figure 27: Successful execution of `hadoop-test.jar` generating the verification code.

3 Word Count

3.1 Step 1: Creating the HDFS Directory Structure and Uploading the Input Dataset

The HDFS input directory was created, and `words.txt` was uploaded for the Word Count job.

Commands executed:

```

hdfs dfs -mkdir -p /hcmus/23127516/warmup/input
hdfs dfs -put -f words.txt /hcmus/23127516/warmup/input/
hdfs dfs -ls /hcmus/23127516/warmup/input

```

```

hadoop@VIETBUINAM:~$ hdfs dfs -mkdir -p /hcmus/23127516/warmup/input
hadoop@VIETBUINAM:~$ hdfs dfs -ls
Found 1 items
drwxr-xr-x - hadoop supergroup 0 2026-07-05 18:19 input
hadoop@VIETBUINAM:~$ hdfs dfs -mkdir -p /hcmus/23127516/warmup/output
hadoop@VIETBUINAM:~$ hdfs dfs -put -f words.txt /hcmus/23127516/warmup/input/
hadoop@VIETBUINAM:~$ hdfs dfs -ls /hcmus/23127516/warmup/input
Found 1 items
-rw-r--r-- 1 hadoop supergroup 4862985 2026-07-06 22:15 /hcmus/23127516/warmup/input/words.txt

```

Figure 28: Creating the HDFS directory structure and uploading the input dataset words.txt.

3.2 Step 2: Compiling the Scala Application

WordCount.scala was compiled with the Hadoop classpath and packaged into WordCount.jar.

Commands executed:

```

scalac -classpath "$(hadoop_classpath)" WordCount.scala
jar cvf WordCount.jar WordCount*.class

```

```

hadoop@VIETBUINAM:~$ nano WordCount.scala
hadoop@VIETBUINAM:~$ scalac -classpath "$(hadoop_classpath)" WordCount.scala
hadoop@VIETBUINAM:~$ jar cvf WordCount.jar WordCount*.class
added manifest
adding: WordCount$.class(in = 2726) (out= 1384)(deflated 49%)
adding: WordCount$IntSumReducer.class(in = 2189) (out= 907)(deflated 58%)
adding: WordCount$TokenizerMapper$anonfun$map$1.class(in = 2541) (out= 1255)(deflated 50%)
adding: WordCount$TokenizerMapper.class(in = 2553) (out= 991)(deflated 61%)
adding: WordCount.class(in = 1621) (out= 1257)(deflated 22%)

```

Figure 29: Successful compilation and packaging of the Scala application into a JAR file.

3.3 Step 3: Submitting the Job to YARN and Resolving the ClassNotFoundException

The first MapReduce submission failed inside the YARN containers with the following exception:

Error: java.lang.ClassNotFoundException: scala.Function1

```

hadoop@VIETBUINAM:~$ hadoop jar WordCount.jar WordCount /hcmus/23127516/warmup/input/words.txt /hcmus/23127516/warmup/output
2026-07-06 22:36:34,401 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-07-06 22:36:35,753 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-07-06 22:36:35,770 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_1783350631818_0004
2026-07-06 22:36:35,940 INFO input.FileInputFormat: Total input files to process : 1
2026-07-06 22:36:35,994 INFO mapreduce.JobSubmitter: number of splits:1
2026-07-06 22:36:36,110 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1783350631818_0004
2026-07-06 22:36:36,111 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-07-06 22:36:36,245 INFO conf.Configuration: resource-types.xml not found
2026-07-06 22:36:36,246 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-07-06 22:36:36,303 INFO impl.YarnClientImpl: Submitted application application_1783350631818_0004
2026-07-06 22:36:36,328 INFO mapreduce.Job: The url to track the job: http://VIETBUINAM.localdomain:8088/proxy/application_1783350631818_0004/
2026-07-06 22:36:36,329 INFO mapreduce.Job: Running job: job_1783350631818_0004
2026-07-06 22:36:40,390 INFO mapreduce.Job: Job job_1783350631818_0004 running in uber mode : false
2026-07-06 22:36:40,391 INFO mapreduce.Job: map 0% reduce 0%
2026-07-06 22:36:43,426 INFO mapreduce.Job: Task Id : attempt_1783350631818_0004_m_000000_0, Status : FAILED
Error: java.lang.ClassNotFoundException: scala.Function1
    at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(BuiltinClassLoader.java:641)
    at java.base/jdk.internal.loader.ClassLoaders$AppClassLoader.loadClass(ClassLoaders.java:188)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:525)
    at java.base/java.lang.Class.forName0(Native Method)
    at java.base/java.lang.Class.forName(Class.java:469)
    at org.apache.hadoop.conf.Configuration.getClassByNameOrNull(Configuration.java:2661)
    at org.apache.hadoop.conf.Configuration.getClassByName(Configuration.java:2626)
    at org.apache.hadoop.conf.Configuration.getClass(Configuration.java:2722)
    at org.apache.hadoop.mapreduce.task.JobContextImpl.getMapperClass(JobContextImpl.java:189)
    at org.apache.hadoop.mapred.MapTask.runNewMapper(MapTask.java:761)
    at org.apache.hadoop.mapred.MapTask.run(MapTask.java:349)
    at org.apache.hadoop.mapred.YarnChild$2.run(YarnChild.java:178)
    at java.base/java.security.AccessController.doPrivileged(AccessController.java:712)
    at java.base/javax.security.auth.Subject.doAs(Subject.java:439)
    at org.apache.hadoop.security.authentication.util.SubjectUtil.doAs(SubjectUtil.java:328)
    at org.apache.hadoop.security.UserGroupInformation.doAs(UserGroupInformation.java:1958)
    at org.apache.hadoop.mapred.YarnChild.main(YarnChild.java:172)

```

Figure 30: System log reporting the missing Scala runtime library.

Analysis

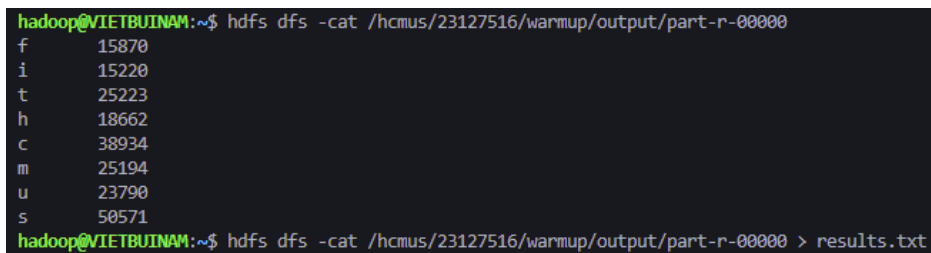
The `hadoop jar` command loaded the Hadoop libraries, but the YARN container JVMs did not include the Scala runtime. As a result, the containers could not find `scala.Function1`, which is provided by `scala-library.jar`, and the Mapper tasks failed.

Solution

The Scala runtime path `/usr/share/scala-2.11/lib/*` was added to `yarn.application.classpath` in `yarn-site.xml`. The incomplete output directory was then removed, and YARN was restarted to apply the configuration.

Commands executed:

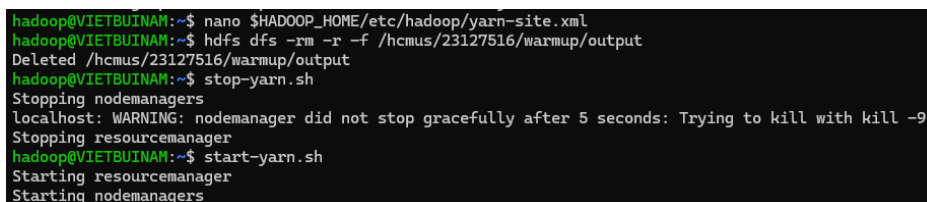
```
nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
```



```
hadoop@VIETBUINAM:~$ hdfs dfs -cat /hcmus/23127516/warmup/output/part-r-00000
f      15870
i      15220
t      25223
h      18662
c      38934
m      25194
u      23790
s      50571
hadoop@VIETBUINAM:~$ hdfs dfs -cat /hcmus/23127516/warmup/output/part-r-00000 > results.txt
```

Figure 31: Adding the Scala runtime library path to the YARN classpath in `yarn-site.xml`.

```
hdfs dfs -rm -r -f /hcmus/23127516/warmup/output
stop-yarn.sh && start-yarn.sh
```



```
hadoop@VIETBUINAM:~$ nano $HADOOP_HOME/etc/hadoop/yarn-site.xml
hadoop@VIETBUINAM:~$ hdfs dfs -rm -r -f /hcmus/23127516/warmup/output
Deleted /hcmus/23127516/warmup/output
hadoop@VIETBUINAM:~$ stop-yarn.sh
Stopping nodemanagers
localhost: WARNING: nodemanager did not stop gracefully after 5 seconds: Trying to kill with kill -9
Stopping resourcemanager
hadoop@VIETBUINAM:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
```

Figure 32: Removing the incomplete output directory and restarting the YARN cluster.

3.4 Step 4: Successful Job Execution

After the YARN classpath was updated, the MapReduce application was submitted again.

Command executed:

```
hadoop jar WordCount.jar WordCount \
/hcmus/23127516/warmup/input/words.txt \
/hcmus/23127516/warmup/output
```

```

hadoop@VIETBUINAM:~$ hadoop jar WordCount.jar WordCount /hcmus/23127516/warmup/input/words.txt /hcmus/
23127516/warmup/output
2026-07-06 22:40:18,107 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager
at /0.0.0.0:8032
2026-07-06 22:40:19,560 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not per
formed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-07-06 22:40:19,575 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/ha
dooop-yarn/staging/hadoop/.staging/job_1783352407947_0001
2026-07-06 22:40:19,771 INFO input.FileInputFormat: Total input files to process : 1
2026-07-06 22:40:19,820 INFO mapreduce.JobSubmitter: number of splits:1
2026-07-06 22:40:19,920 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1783352407947_0001
2026-07-06 22:40:19,920 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-07-06 22:40:20,056 INFO conf.Configuration: resource-types.xml not found
2026-07-06 22:40:20,057 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-07-06 22:40:20,404 INFO impl.YarnClientImpl: Submitted application application_1783352407947_0001
2026-07-06 22:40:20,431 INFO mapreduce.Job: The url to track the job: http://VIETBUINAM.localdomain:80
88/proxy/application_1783352407947_0001/
2026-07-06 22:40:20,432 INFO mapreduce.Job: Running job: job_1783352407947_0001
2026-07-06 22:40:25,514 INFO mapreduce.Job: Job job_1783352407947_0001 running in uber mode : false
2026-07-06 22:40:25,515 INFO mapreduce.Job: map 0% reduce 0%
2026-07-06 22:40:30,569 INFO mapreduce.Job: map 100% reduce 0%
2026-07-06 22:40:32,617 INFO mapreduce.Job: map 100% reduce 100%
2026-07-06 22:40:32,626 INFO mapreduce.Job: Job job_1783352407947_0001 completed successfully
2026-07-06 22:40:32,678 INFO mapreduce.Job: Counters: 54
  File System Counters
    FILE: Number of bytes read=70
    FILE: Number of bytes written=635715
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=4863109
    HDFS: Number of bytes written=64
    HDFS: Number of read operations=8
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=2
    HDFS: Number of bytes read erasure-coded=0
  Job Counters
    Launched map tasks=1
    Launched reduce tasks=1
    Data-local map tasks=1
    Total time spent by all maps in occupied slots (ms)=2301
    Total time spent by all reduces in occupied slots (ms)=-217
    Total time spent by all map tasks (ms)=2301
    Total time spent by all reduce tasks (ms)=-217
    Total vcore-milliseconds taken by all map tasks=2301
    Total vcore-milliseconds taken by all reduce tasks=-217
    Total megabyte-milliseconds taken by all map tasks=2356224
    Total megabyte-milliseconds taken by all reduce tasks=-222208

```

Figure 33: Successful completion of the MapReduce job (map 100%, reduce 100%).

Result

The MapReduce job completed successfully. The HDFS output was exported to the local machine as the TSV-formatted file `results.txt`.

```

hadoop@VIETBUINAM:~$ hdfs dfs -cat /hcmus/23127516/warmup/output/part-r-00000
f      15870
i      15220
t      25223
h      18662
c      38934
m      25194
u      23790
s      50571
hadoop@VIETBUINAM:~$ hdfs dfs -cat /hcmus/23127516/warmup/output/part-r-00000 > results.txt

```

Figure 34: Exporting the output results.